Core Concepts: Graph-Based Conversational Agents

> Part of the Conversational Agent curriculum. This document covers what "agent" means here, why a graph rather than a plain loop, nodes/edges/conditional routing, state schema and reducers, the tool-calling loop, and checkpointer-backed memory.

1. What "agent" means here

An **agent**, in this sense, is an LLM given two things a single fixed prompt→response call doesn't have: a set of callable **tools**, and the autonomy to decide, per turn, whether and which tools to call and when it has enough information to stop. The defining loop:

1. The model reasons over the conversation so far.
2. It either produces a final answer, or requests a tool call with specific arguments.
3. If it requested a tool call, that tool actually runs, and its result is fed back into the model's context as a new message.
4. Repeat from step 1 until the model produces a final answer (or a bound described in Tool Calling and Guardrails, §3 is hit).

This is a fundamentally different shape from a single-shot "summarize this text" call — the number of model invocations per user turn isn't fixed in advance, and the model's own output partially determines what happens next.

2. Why a graph, not just a while-loop

A bare `while` loop around steps 1–4 above is enough for the simplest possible agent, but real agents quickly need more than "call tools until done": classifying a request before deciding whether to enter the tool-calling loop at all, branching to entirely different handling for different classes of request, doing real work that isn't a tool call (a structured-output classification step, a final response-composition step), and streaming progress partway through. Modeling all of that as nested conditionals inside one function gets unreadable fast.

A **directed-graph orchestration model** makes each of those an explicit **node** and **edge** instead — the control flow is legible by looking at the graph's structure, not by reading through branching logic line by line. **LangGraph** (this curriculum's example framework) is built around exactly this idea; other agent-orchestration frameworks use the same underlying graph-of-steps concept even where the API differs.

3. Nodes, edges, and conditional routing

```python
from langgraph.graph import StateGraph, END

def classify(state): ...
def handle_tools(state): ...
def finalize(state): ...

def route_after_classify(state):
    return "handle_tools" if state.needs_data else "finalize"

graph = StateGraph(AgentState)
graph.add_node("classify", classify)
graph.add_node("handle_tools", handle_tools)
graph.add_node("finalize", finalize)
graph.set_entry_point("classify")
graph.add_conditional_edges("classify", route_after_classify)
graph.add_edge("handle_tools", "finalize")
graph.add_edge("finalize", END)

app = graph.compile()
```

- A **node** is a plain function (or a callable object) that reads the current state and returns a partial update to it — nothing more magical than that.
- An **edge** decides which node runs next; a **conditional edge** picks between several possible next nodes based on the state a node just produced (`route_after_classify` above).
- Not every node calls an LLM — some are pure, deterministic Python with no model call at all. This distinction matters in practice: LLM-calling nodes cost money, add latency, and are non-deterministic; plain-Python nodes are none of those, and it's worth knowing, node by node, which kind you're looking at.

4. State schema and reducers

The graph's **state** is one typed schema shared across every node — commonly a `TypedDict`, but an ordinary Pydantic `BaseModel` works too in frameworks like LangGraph (don't assume a framework only supports the shape its quick-start examples happen to show; check). Each node returns a **partial** update — only the fields it actually changed — which the framework merges into the running state.

A **reducer** controls *how* a specific field's updates get merged, instead of the default "just overwrite it": the canonical example is a message-history field using an append-only reducer, so that every node that produces a message adds to the conversation history instead of replacing it outright.

```python
from typing import Annotated
from pydantic import BaseModel
from langgraph.graph.message import add_messages

class AgentState(BaseModel):
    messages: Annotated[list, add_messages]   # append-only reducer, not a plain overwrite
    current_query: str
```

Use the framework's own purpose-built reducer for a shape this common (`add_messages` above) rather than hand-rolling equivalent merge logic — and **verify the merge behavior by actually invoking the compiled graph**, not by constructing two state objects and comparing them by hand. A reducer's behavior is a property of running the graph through the framework's own update machinery, not of the state class in isolation — see Testing, §4 for what that test actually needs to look like.

5. The tool-calling loop, concretely

Binding tools to a model means passing it a list of tool schemas alongside the prompt; a tool-calling-capable LLM API can then respond with either plain text, or a structured "call this tool with these arguments" request (or several, in one turn). A dedicated **tools node** executes whatever was requested, appends each result back into the message history as a tool-result message, and hands control back to the model node.

This model-node ↔ tools-node cycle is the **one real loop** in an otherwise mostly-linear graph — everything else in a typical agent graph fires at most once per turn by construction. Because it's the one place that can genuinely run more than once, it's also the one place that needs an explicit exit condition beyond "the model said it's done": see Tool Calling and Guardrails, §3 for why a hard cap matters even when the model is well-behaved almost all the time.

6. Memory: checkpointers and thread-scoped state

A **checkpointer** persists a graph's state between separate invocations, keyed by a **thread id** — this is the mechanism that gives a multi-turn conversation actual memory of earlier turns, without the caller having to re-send the full conversation history on every request.

```python
result = app.invoke(
    {"current_query": "and what about India?"},   # a PARTIAL update, not a fresh AgentState(...)
    config={"configurable": {"thread_id": "session-abc123"}},
)
```

Always invoke with a partial update, never a freshly constructed full state object. A fresh `AgentState(...)` still type-checks and runs without error — it just silently overwrites *every* field the checkpointer was tracking for that thread, including ones meant to persist for the thread's entire lifetime (an accumulated cache, for instance), not just the current turn. This is a genuinely easy mistake precisely because nothing about it looks wrong at a glance; it only shows up as state mysteriously "resetting" between turns.

Know which fields are per-turn and which are per-thread. Because a checkpointer carries *everything* forward, any field meant to describe only the current answer (the widgets to render, the follow-up suggestions, the key-figure cards, the notes to show) will silently leak into the next turn unless a node explicitly resets it at the start of each turn. Fields meant to live for the whole conversation (the message history, a tool-result cache) must be the opposite — never reset. A short, explicit "reset these per-turn fields" step at the top of the graph, plus a test that runs two turns in a row and asserts the second turn doesn't contain the first turn's widgets, catches the most common version of this bug.

An in-memory checkpointer is fine for development and for a single-process deployment; anything that needs to survive a process restart, or run behind multiple server instances, needs a checkpointer backed by external, shared storage instead. Note that "the server forgot the conversation" and "the user's screen forgot the conversation" are two separate problems: a UI that keeps its own copy of the thread (so navigating to another page and back doesn't blank the conversation) is a client-side concern, and it still has to send the same thread id back for the server's memory to line up with what the user sees. Whichever side owns the history, a **reset** action on the client must also abort any request still in flight and ignore its eventual reply, or a slow answer from the discarded conversation lands in the new one.

One more state-schema consequence worth knowing: when a checkpointer serializes state, every custom type stored in it (a record of a tool call, a widget spec, a link, a key-figure card) may need to be explicitly registered with the checkpointer's serializer. An unregistered type often works today with only a logged warning, and is scheduled to become a hard failure in a later framework release — register new state types at the same time as you add them.

7. Vocabulary summary

Term	Meaning
Node	A function reading graph state and returning a partial update
Edge	Fixed or conditional routing to the next node
State schema	The typed shape shared across every node (`TypedDict` or a Pydantic model)
Reducer	Custom merge logic for a specific state field, instead of overwrite-by-default
Tool binding	Passing a model a list of callable tool schemas it can request calls against
Checkpointer	Persists graph state between invocations, keyed by thread id
Thread id	The key that scopes persisted state to one ongoing conversation

See also

- Conversational Agent curriculum index
- Tool Calling, Guardrails and Generative UI — what runs inside the tools node, and how the loop this document introduced gets bounded and rendered
- Testing — verifying reducer and routing behavior against a real graph
- MCP Server, Core Concepts for the tool source this curriculum assumes an agent is calling